

Memory Hierarchy (I)

Outline

- Random-Access Memory (RAM)
- Nonvolatile Memory
- Data transfer between memory and CPU
- Hard Disk
- Data transfer between memory and disk
- SSD
- Storage trends
- Suggested Reading: 6.1

Random-Access Memory (RAM)

- Key features
 - RAM is packaged as a chip.
 - Basic storage unit is a cell (one bit per cell).
 - Multiple RAM chips form a memory.



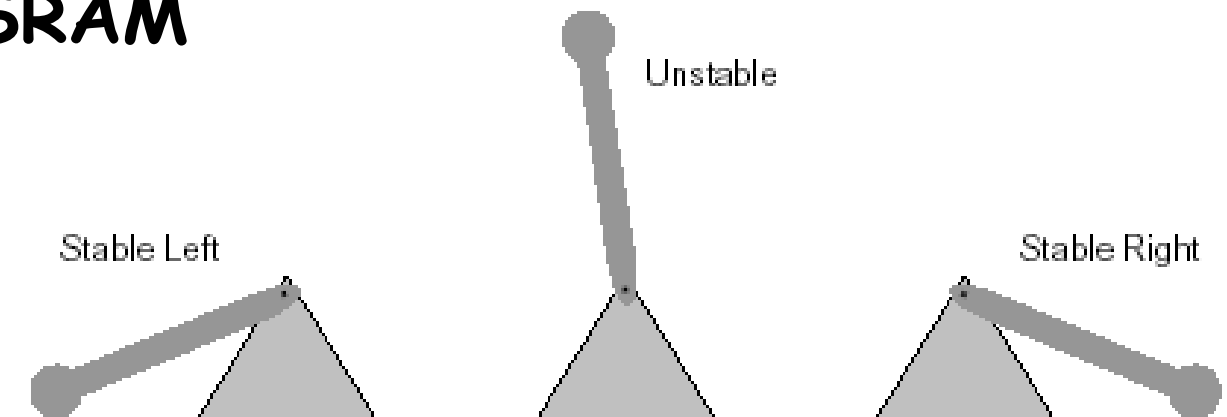
Random-Access Memory (RAM)

- Static RAM (SRAM)
 - Each cell stores bit with a six-transistor circuit.
 - Retains value indefinitely, as long as it is kept powered.
 - Relatively insensitive to disturbances such as electrical noise.
- Dynamic RAM (DRAM)
 - Each cell stores bit with a capacitor and transistor.
 - Value must be refreshed every 10-100 ms.
 - Sensitive to disturbances.

SRAM vs DRAM summary

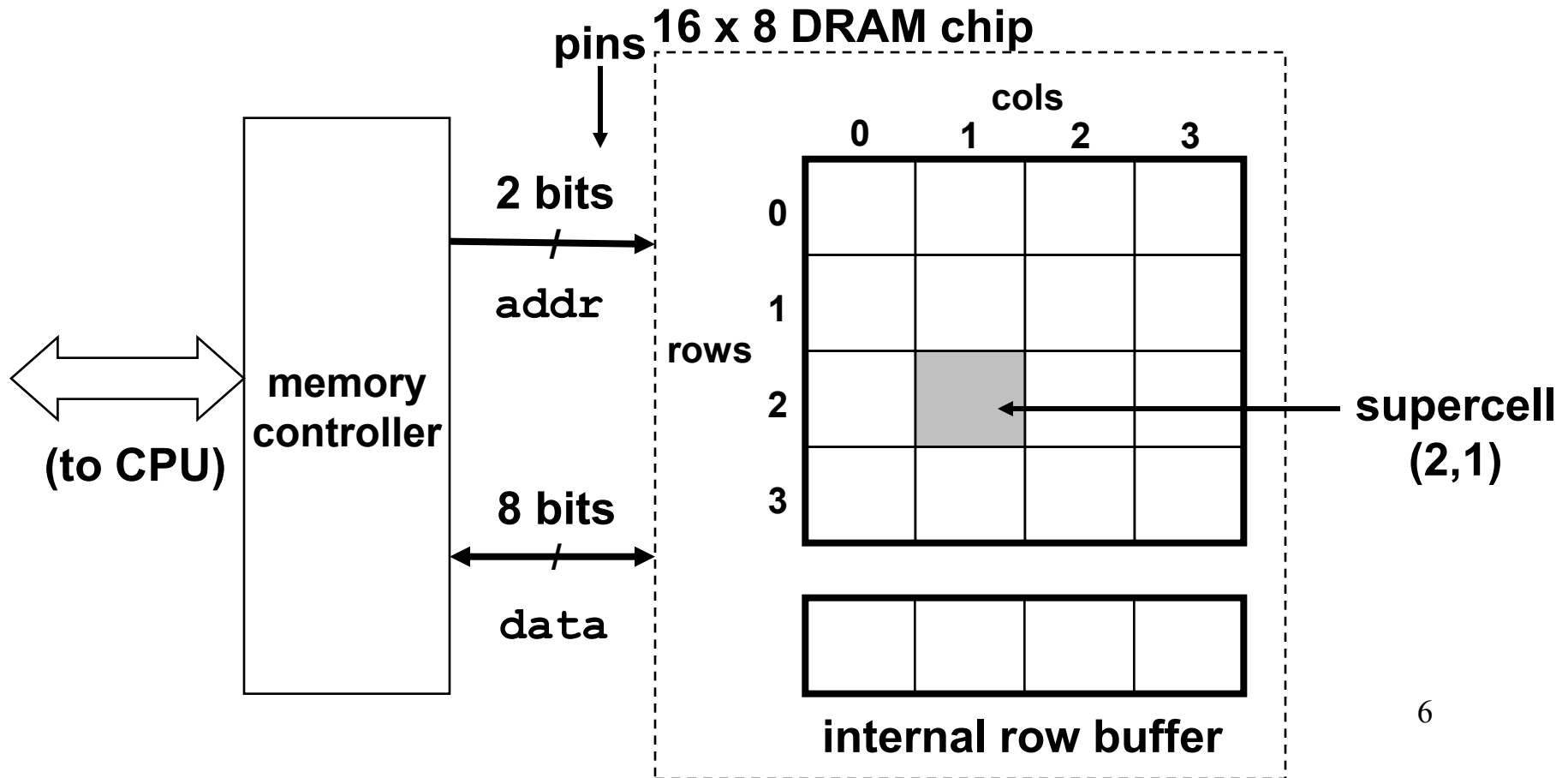
	Tran. per bit	Access time	Persist?	Sensitive?	Cost	Applications
SRAM	6	1X	Yes	No	1000x	cache memories
DRAM	1	10X	No	Yes	1X	Main memories, frame buffers

SRAM



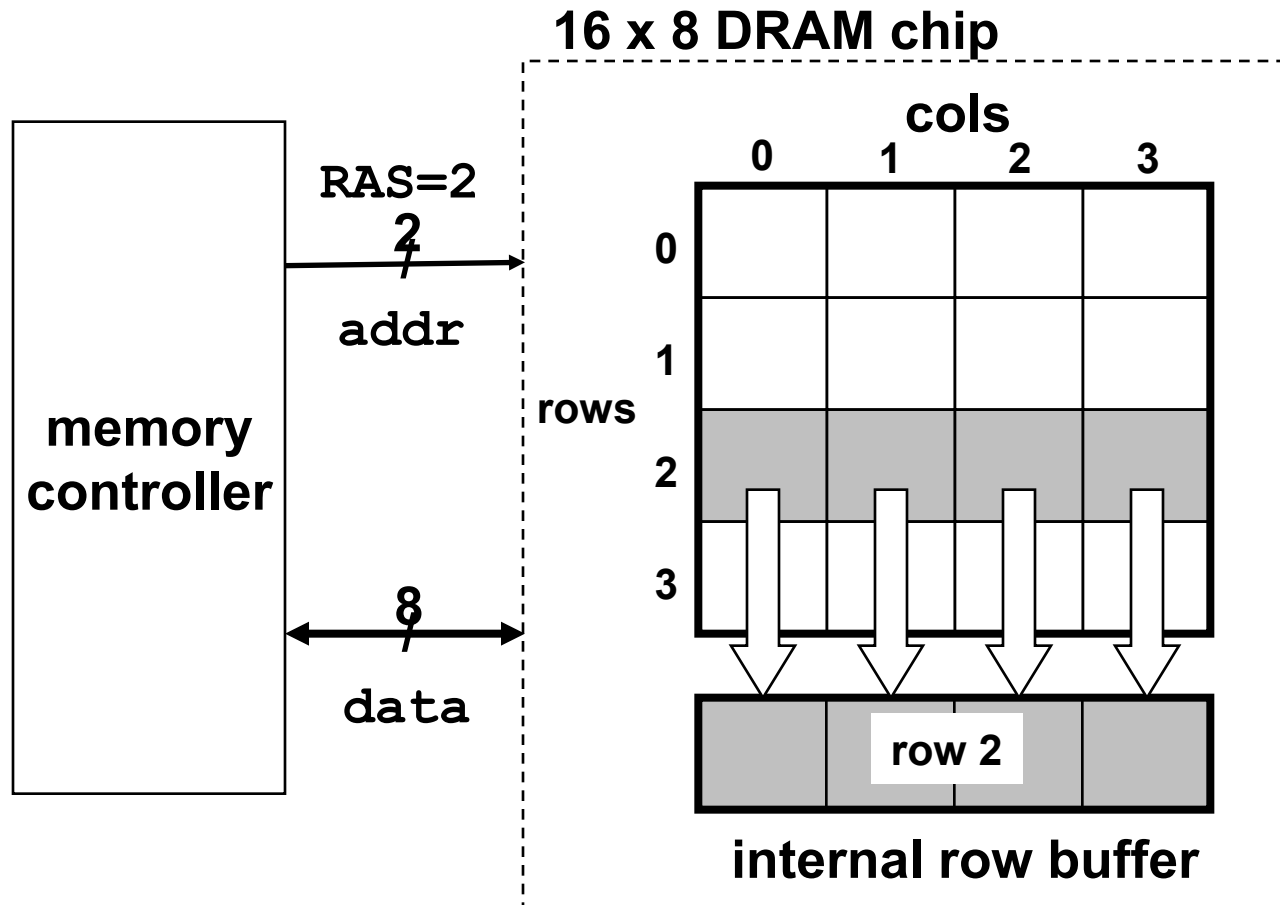
Conventional DRAM organization

- $d \times w$ DRAM:
 - dw total bits organized as d supercells of size w bits



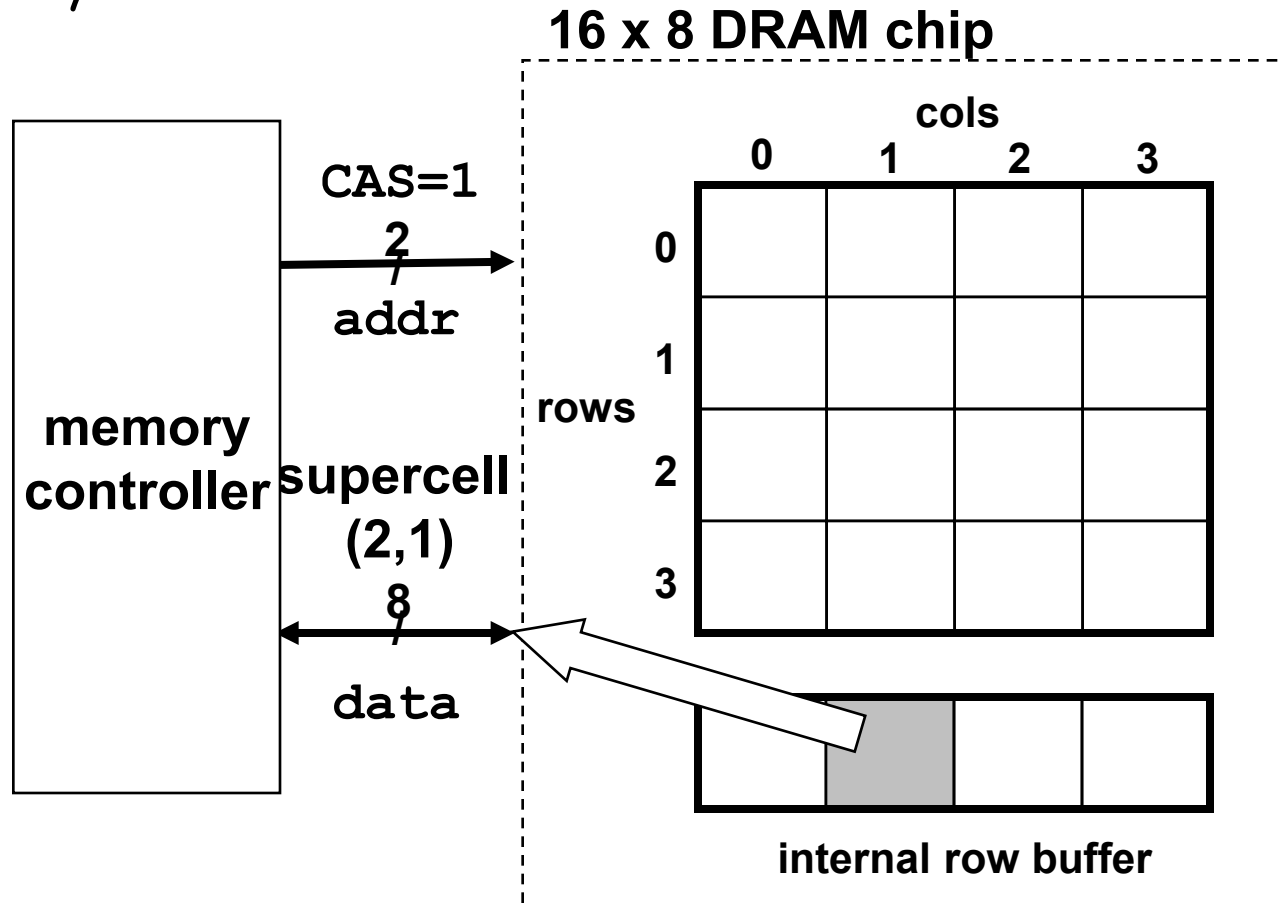
Reading DRAM supercell (2,1)

- Step 1(a): Row access strobe (RAS) selects row 2.
- Step 1(b): Row 2 copied from DRAM array to row buffer.

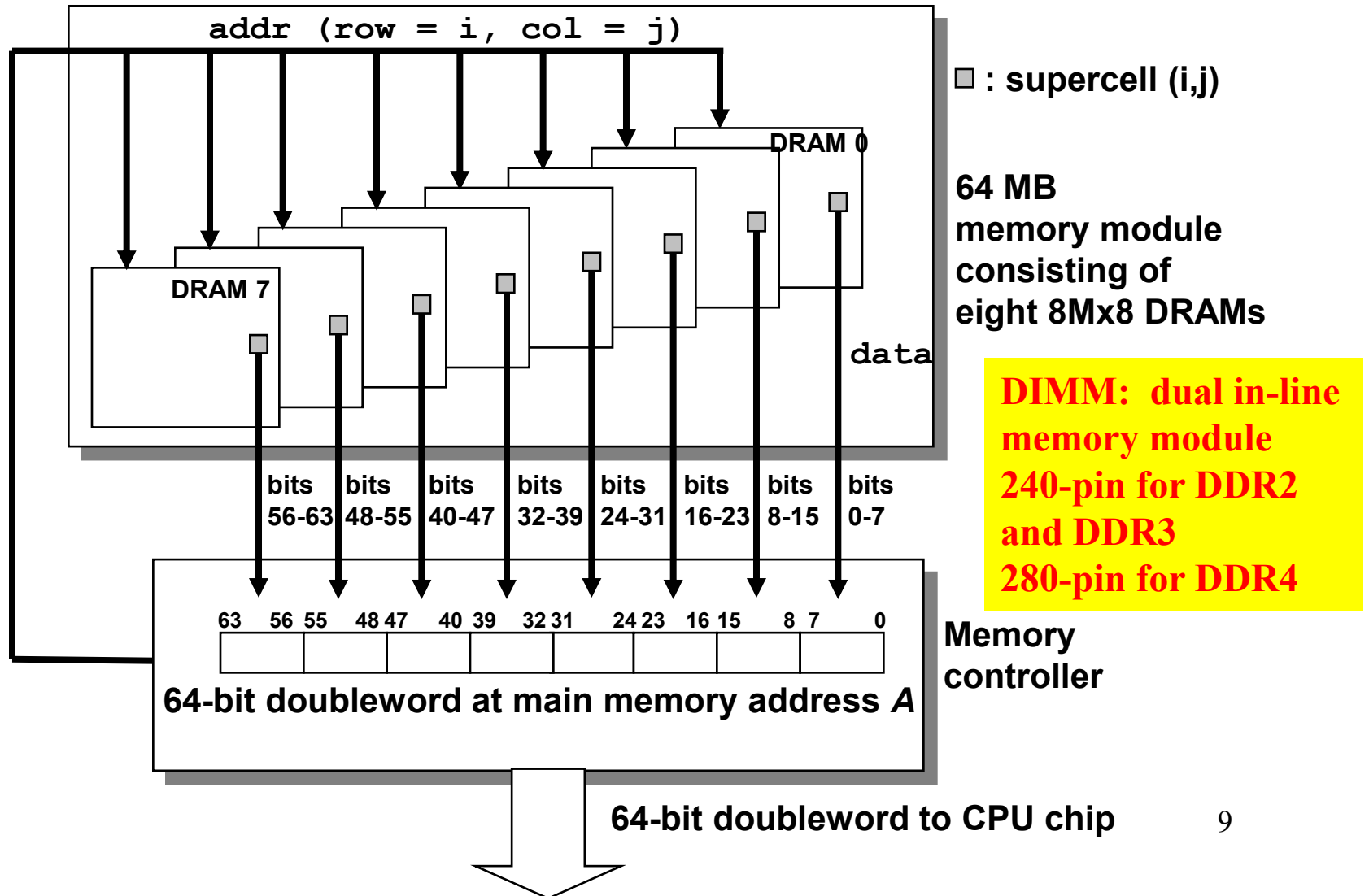


Reading DRAM supercell (2,1)

- Step 2(a): Column access strobe (CAS) selects column 1.
- Step 2(b): Supercell (2,1) copied from buffer to data lines, and eventually back to the CPU.



Memory modules



Nonvolatile memories

- DRAM and SRAM are volatile memories
 - Lose information if powered off.
- Nonvolatile memories retain value even if powered off
 - Generic name is read-only memory (ROM).
 - Misleading because some ROMs can be read and modified.

Nonvolatile memories

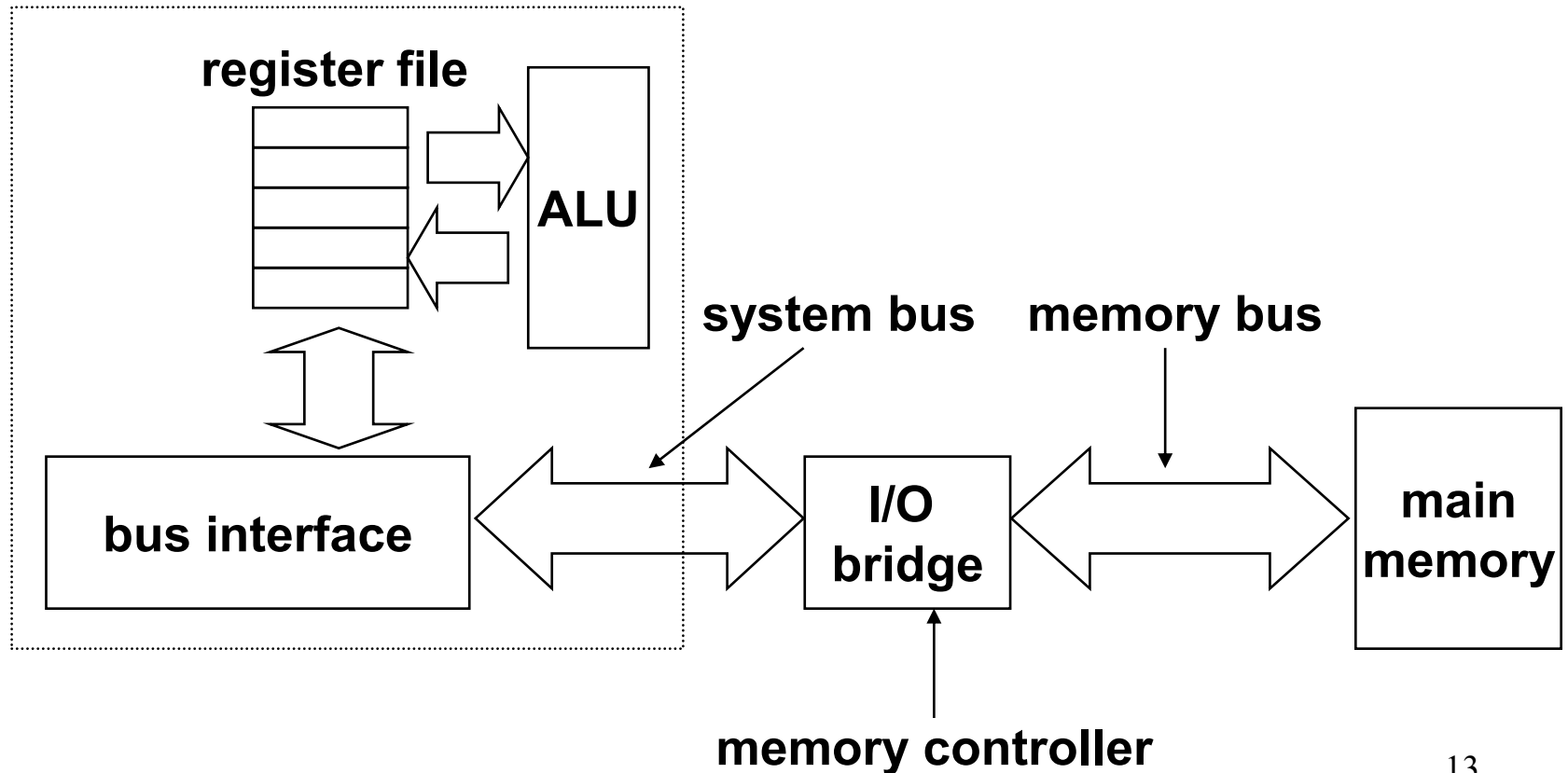
- Firmware
 - Program stored in a ROM
 - BIOS (basic input/output system)
 - Boot time code
 - a small set of primitive input and output functions
 - graphics cards, disk controllers
 - Translate I/O (input/output) requests from the CPU

Bus Structure Connecting CPU and memory

- A **bus** is a collection of parallel wires that carry address, data, and control signals
- Buses are typically shared by multiple devices

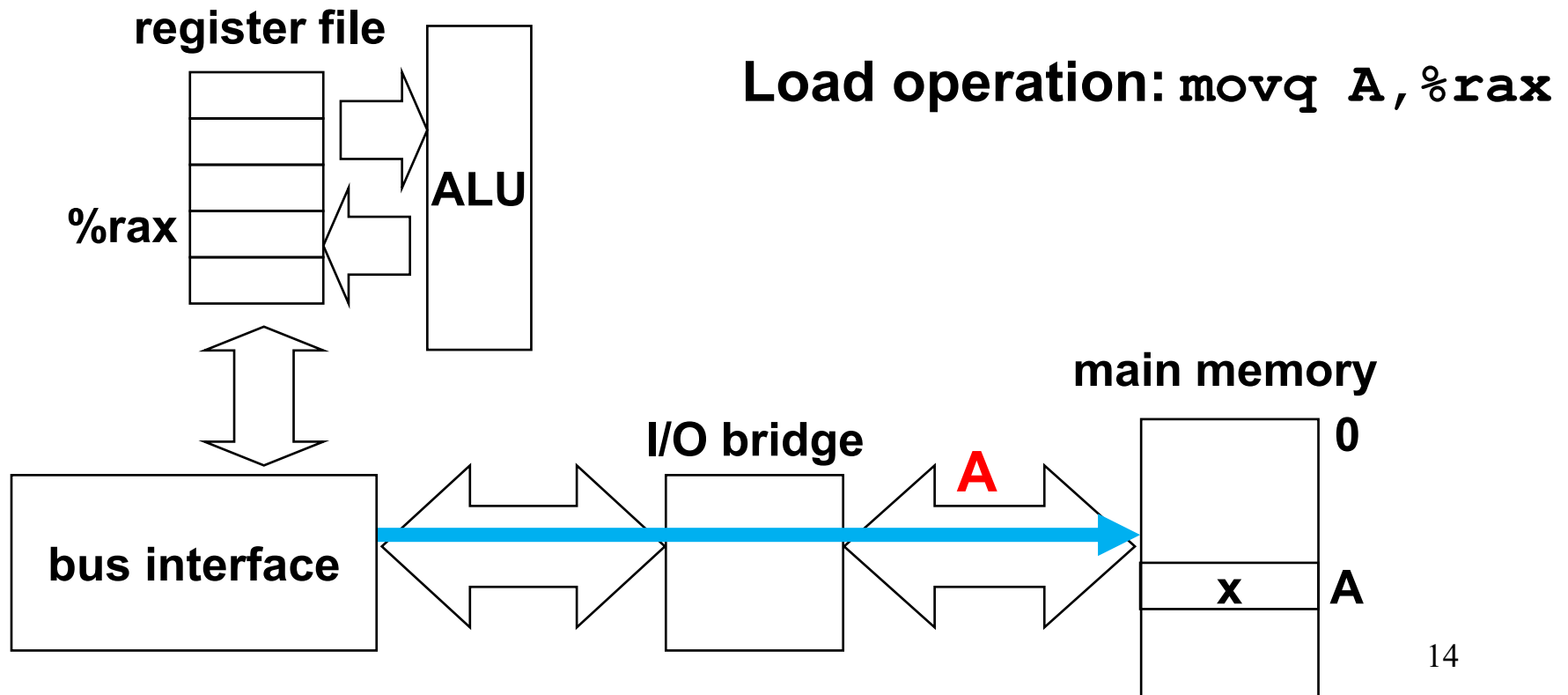
Bus Structure Connecting CPU and memory

CPU chip



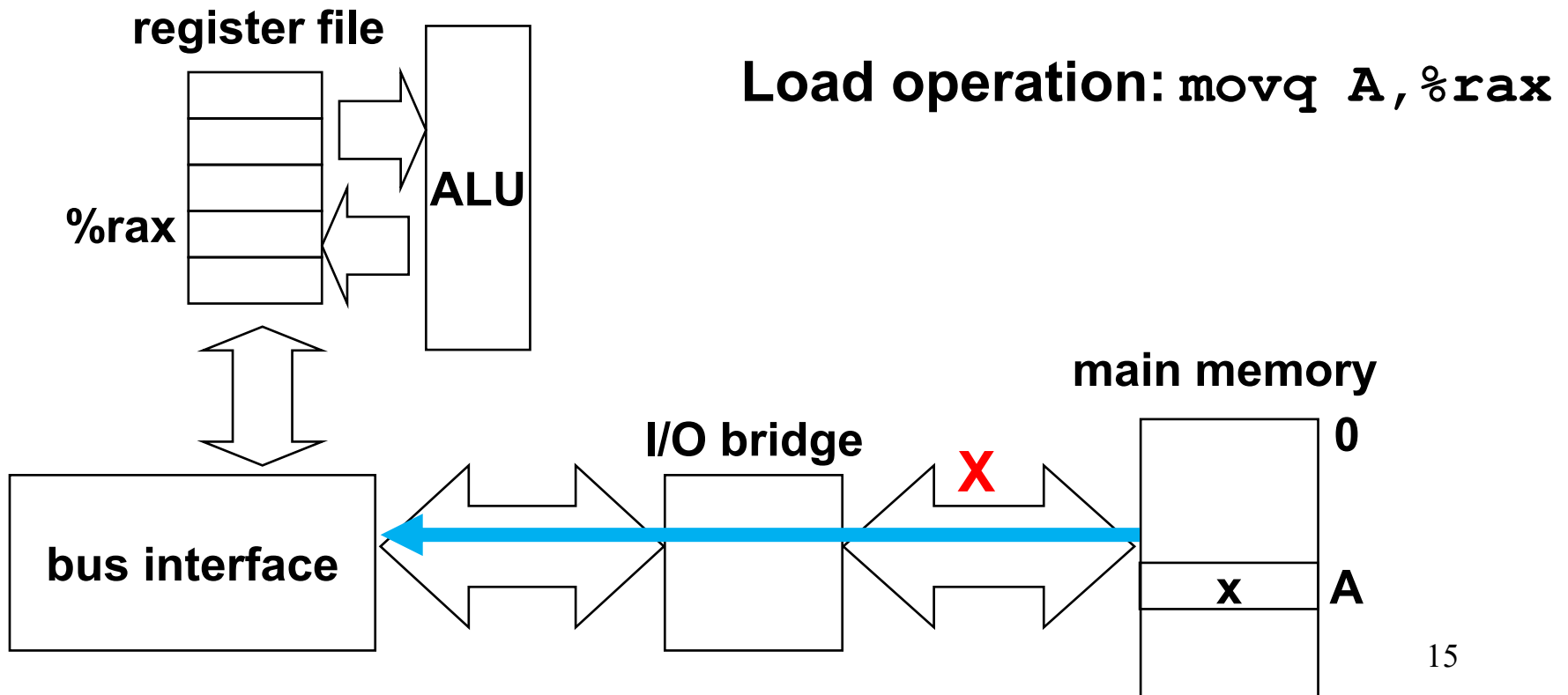
Memory read transaction

1. CPU places address **A** on the memory bus



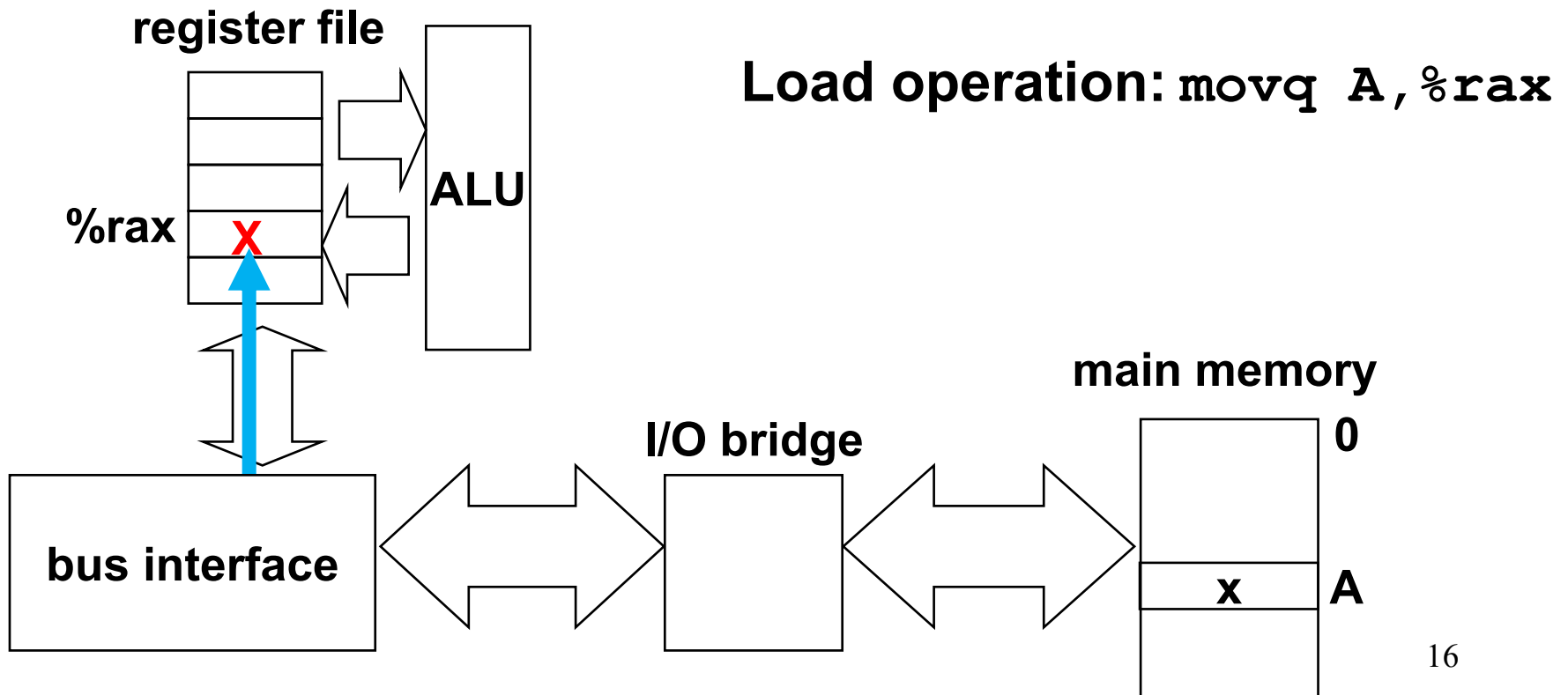
Memory read transaction

2. Main memory reads **A** from the memory bus, retrieves word **x**, and places it on the bus.



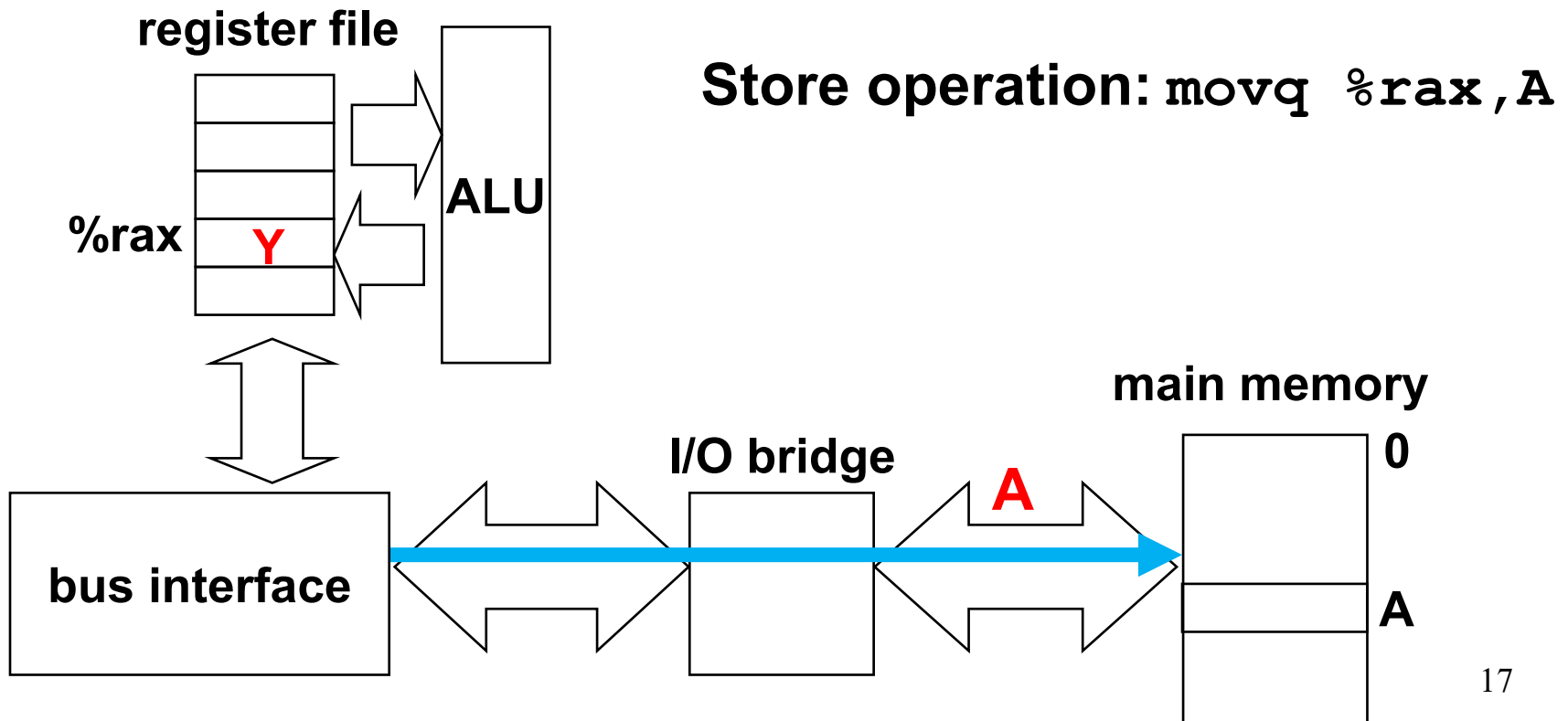
Memory read transaction

3. CPU read word **x** from the bus and copies it into register **%rax**.



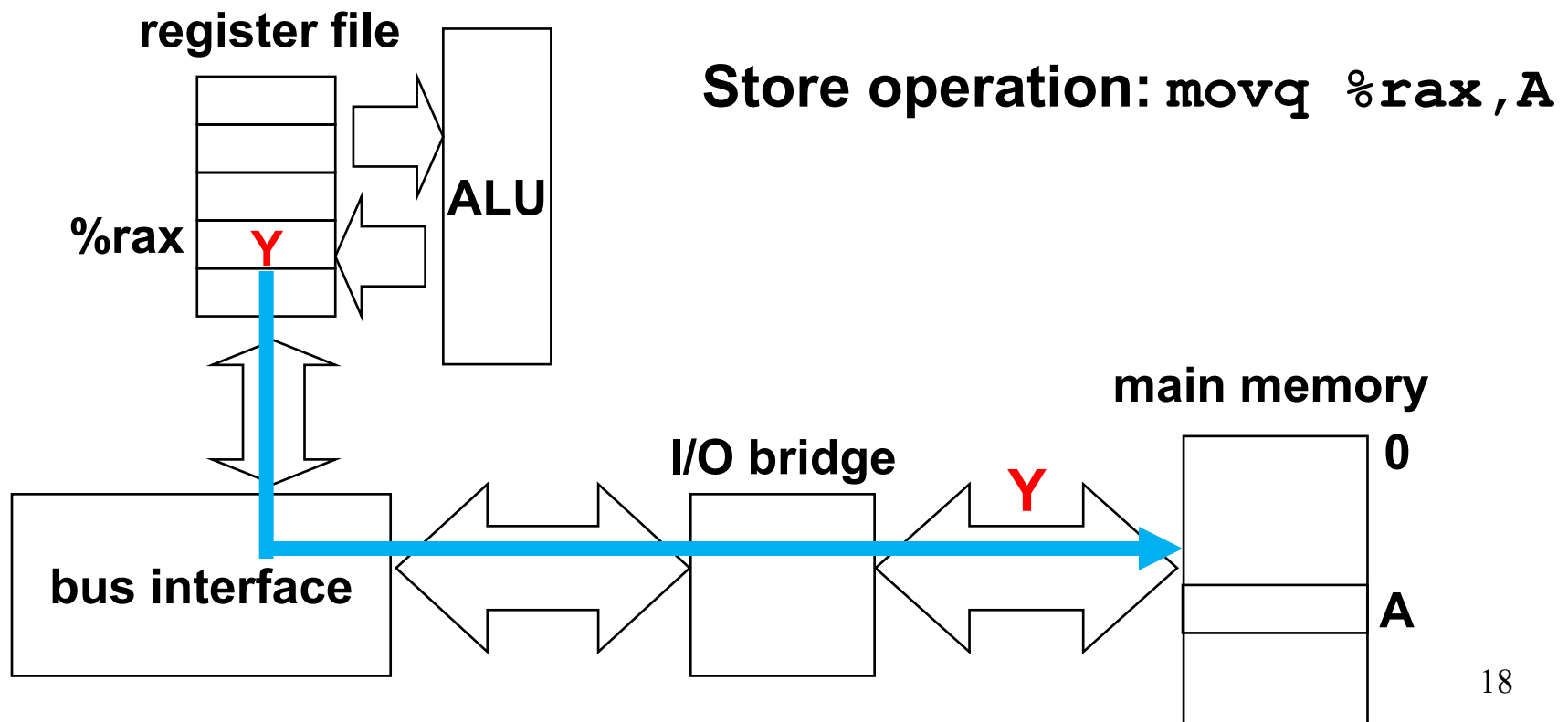
Memory write transaction

1. CPU place address **A** on bus. Main memory reads it and waits for the corresponding data to arrive.



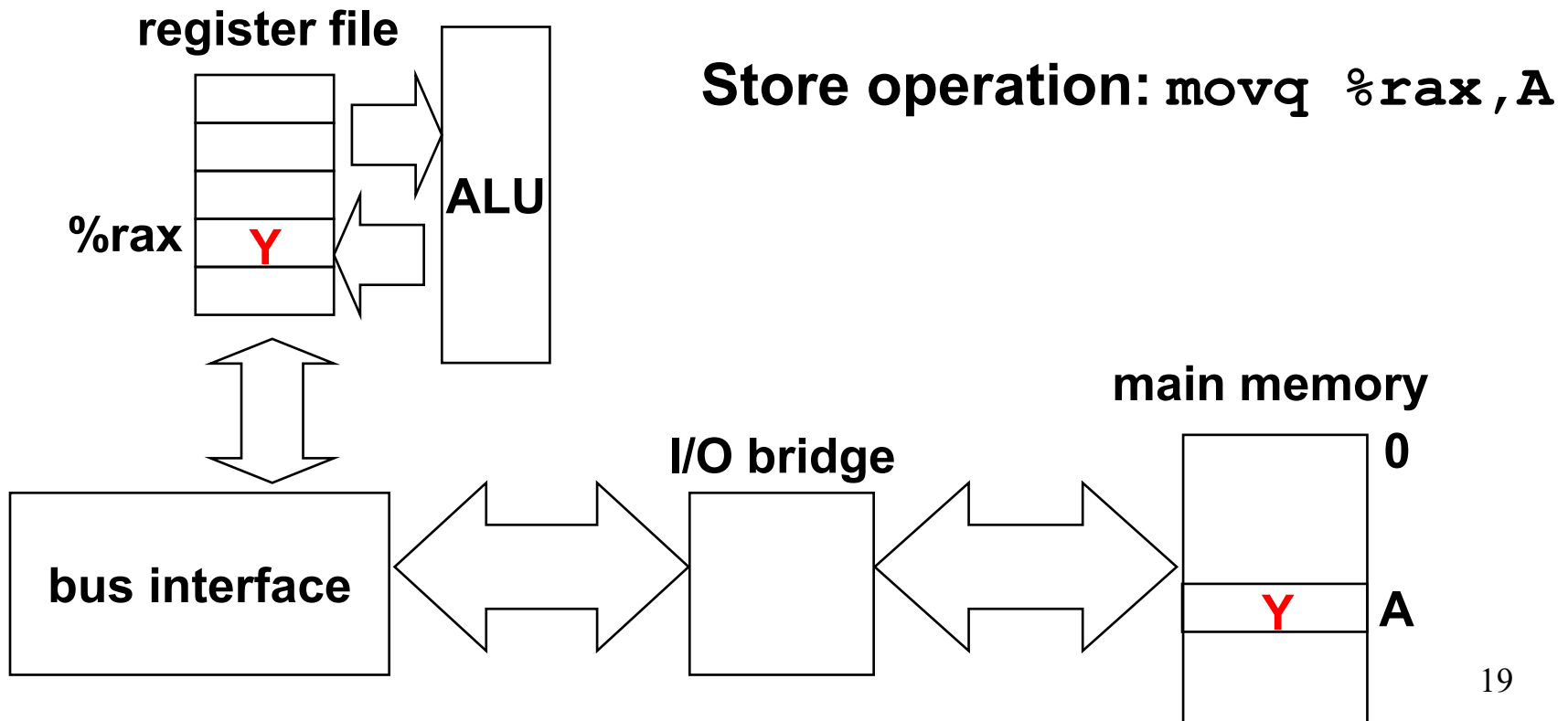
Memory write transaction

2. CPU places data word **Y** on the bus.

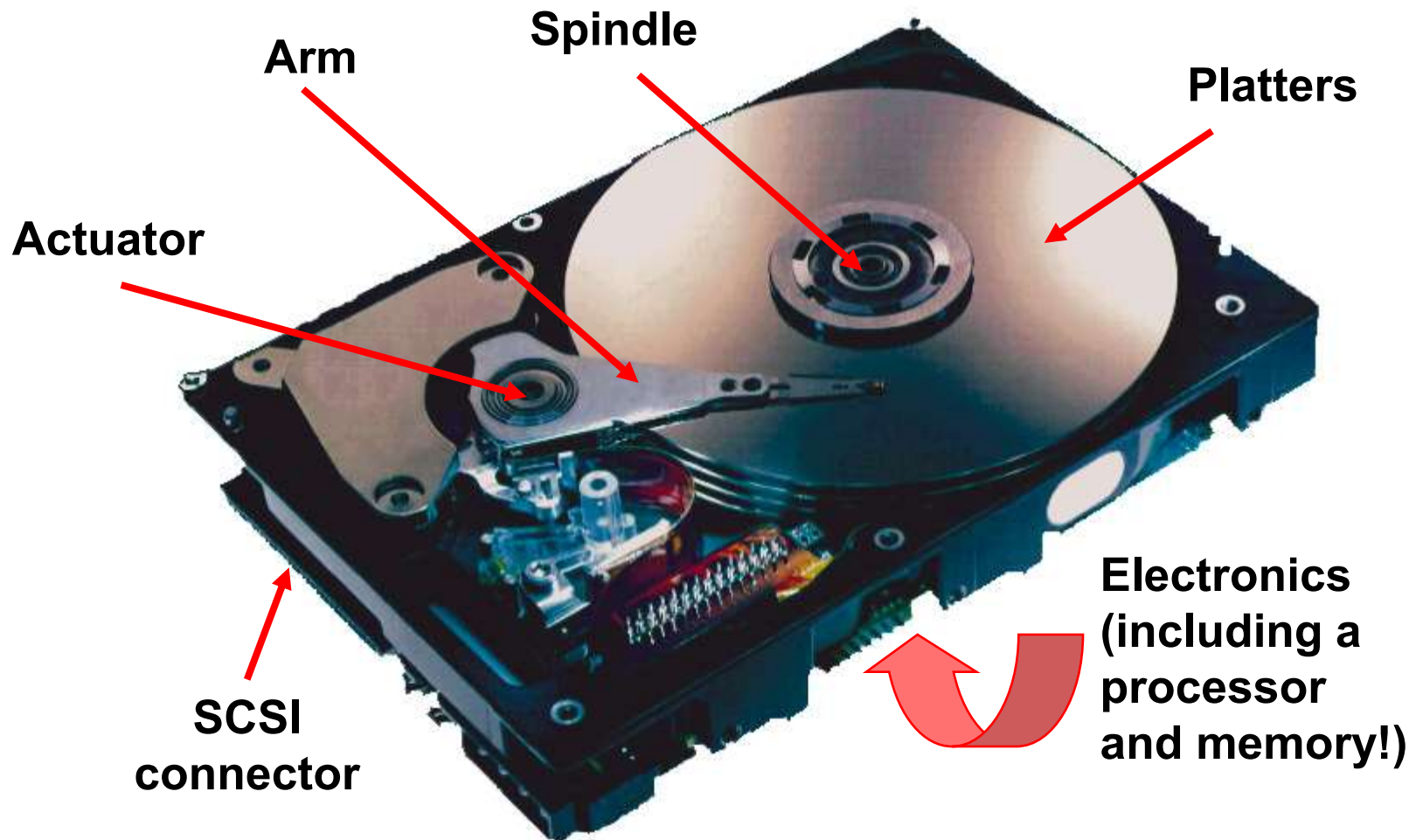


Memory write transaction

3. Main memory read data word **Y** from the bus and stores it at address **A**

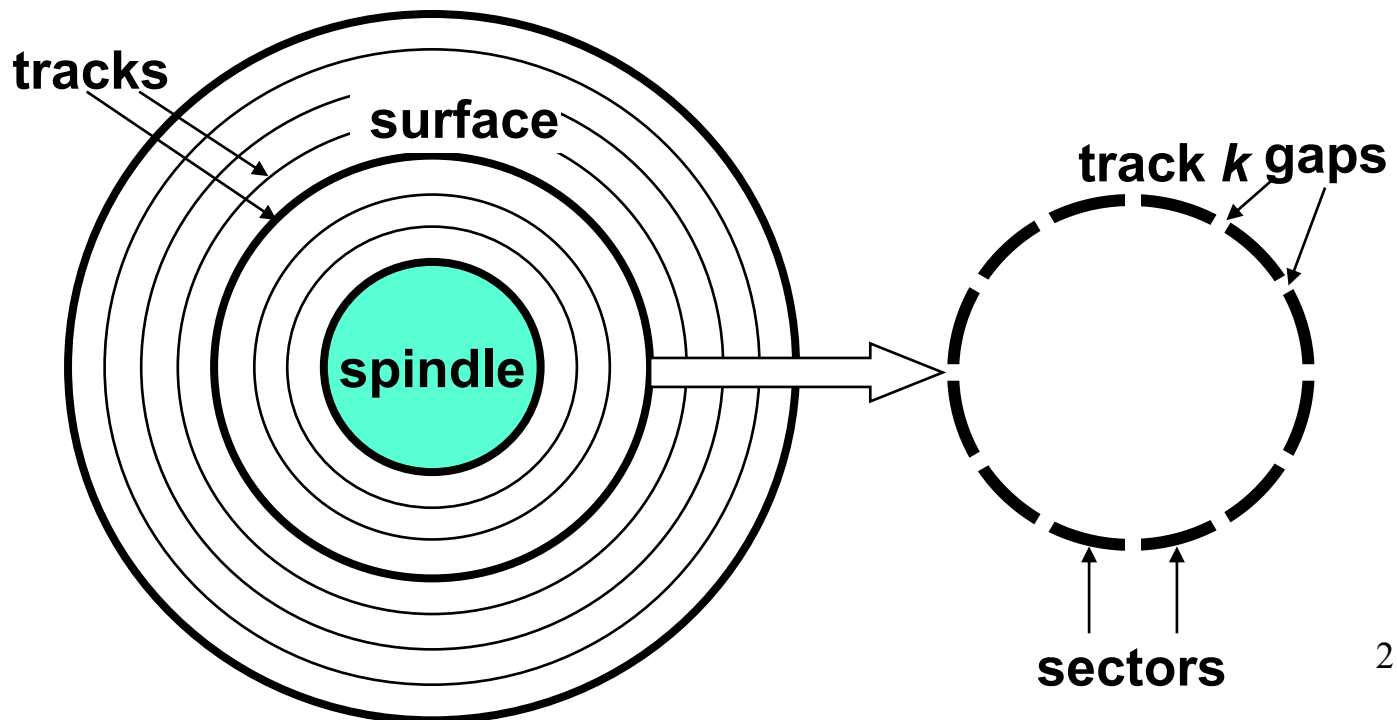


What's inside a disk drive?



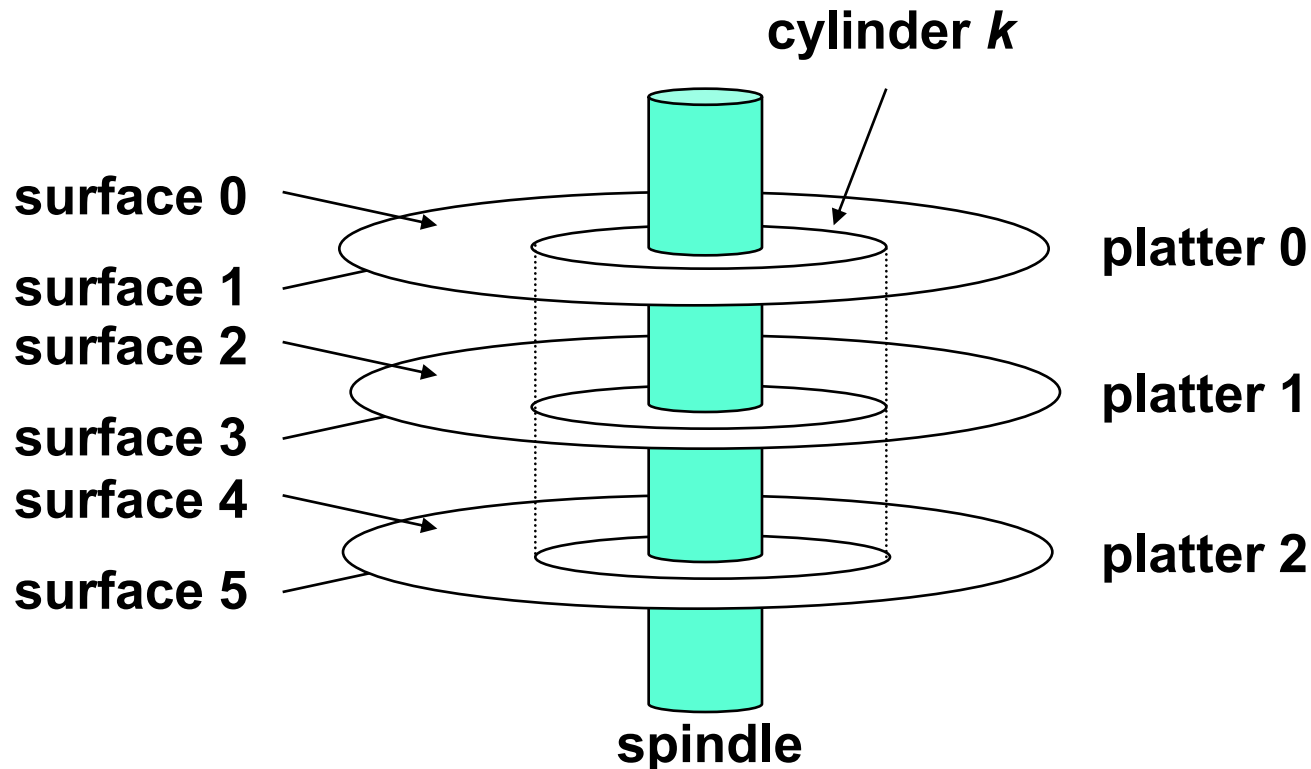
Disk geometry

- Disks consist of **platters**, each with two **surfaces**.
- Each surface consists of concentric rings called **tracks**.
- Each track consists of **sectors** separated by gaps.



Disk geometry (multiple-platter view)

- Aligned tracks form a **cylinder**.



Disk capacity

- Capacity
 - maximum number of bits that can be stored
 - Vendors express capacity in units of gigabytes (GB), where $1 \text{ GB} = 10^9$.
- Capacity is determined by these technology factors:
 - **Recording density** (bits/in): number of bits that can be squeezed into a 1 inch segment of a track.
 - **Track density** (tracks/in): number of tracks that can be squeezed into a 1 inch radial segment.
 - **Areal density** (bits/in²): product of recording and track density.

Disk capacity

- Old fashioned disks
 - Each track has the same number of sectors
- Modern disks partition tracks into disjoint subsets called **recording zones**
 - Each track in a zone has the same number of sectors, determined by the circumference of innermost track
 - Each zone has a different number of sectors/track

Computing disk capacity

```
Capacity = (# bytes/sector)
           x (avg. # sectors/track)
           x (# tracks/surface)
           x (# surfaces/platter)
           x (# platters/disk)
```

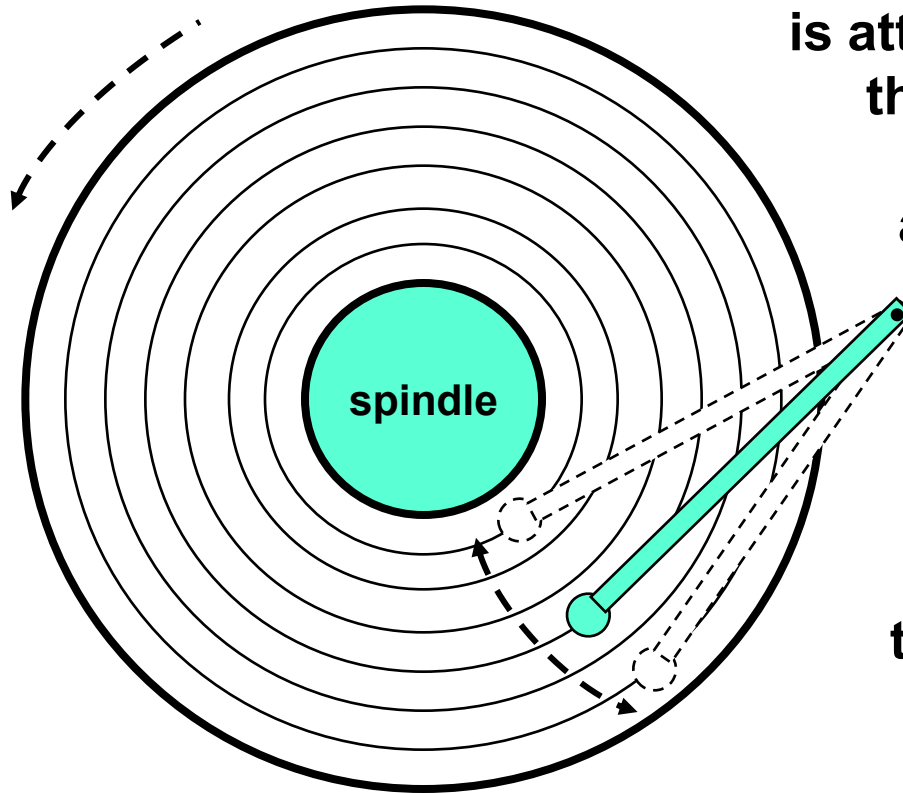
Example:

- 512 bytes/sector
- 300 sectors/track (on average)
- 20,000 tracks/surface
- 2 surfaces/platter
- 5 platters/disk

```
Capacity = 512 x 300 x 20000 x 2 x 5
          = 30,720,000,000
          = 30.72 GB
```

Disk operation (single-platter view)

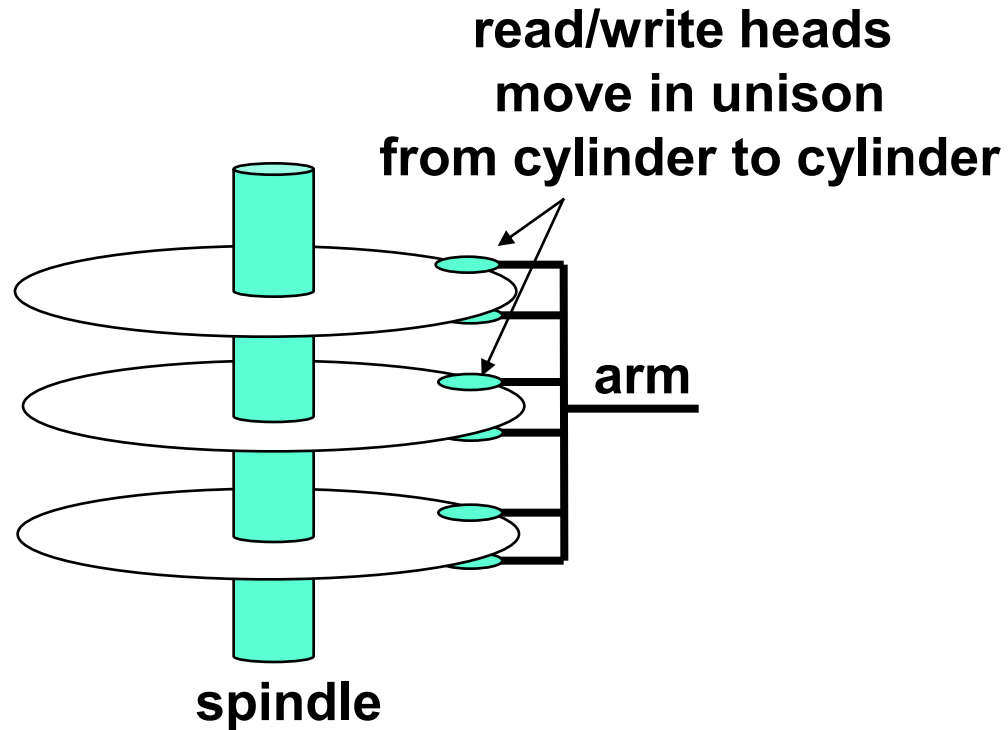
The disk surface spins at a fixed **rotational rate**



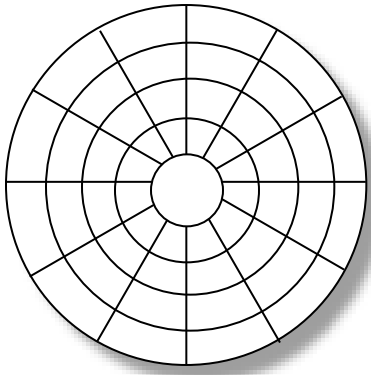
The read/write **head** is attached to the end of the **arm** and flies over the disk surface on a thin cushion of air.

By moving radially, the arm can position the read/write head over any track.

Disk operation (multi-platter view)



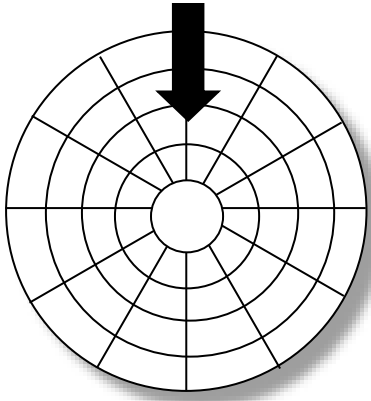
Disk Structure - top view of single platter



Surface organized into tracks

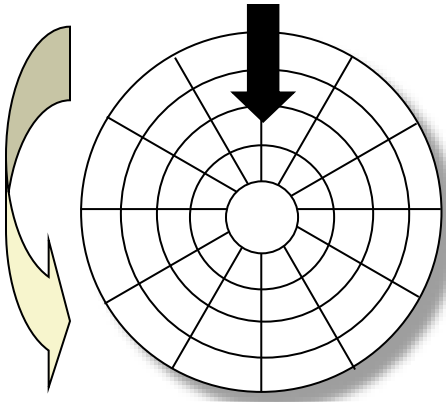
Tracks divided into sectors

Disk Access



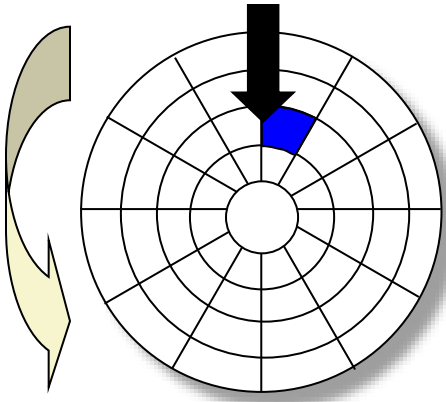
Head in position above a track

Disk Access



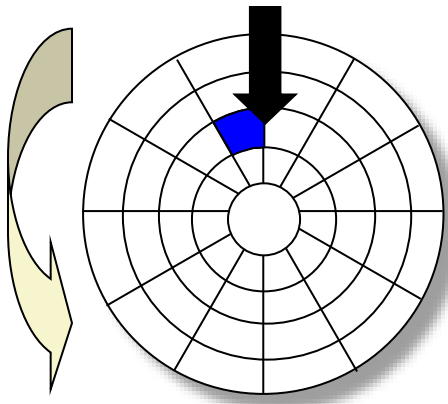
Rotation is counter-clockwise

Disk Access - Read



About to read blue sector

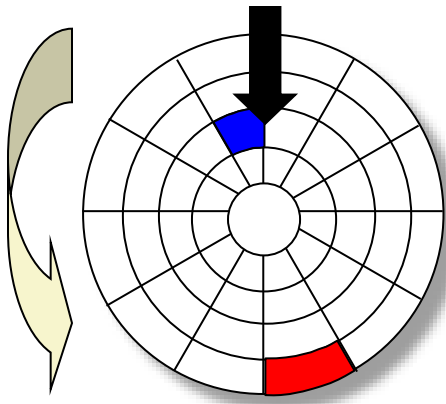
Disk Access - Read



After **BLUE**
read

**After reading blue sector
Read one sector each time!**

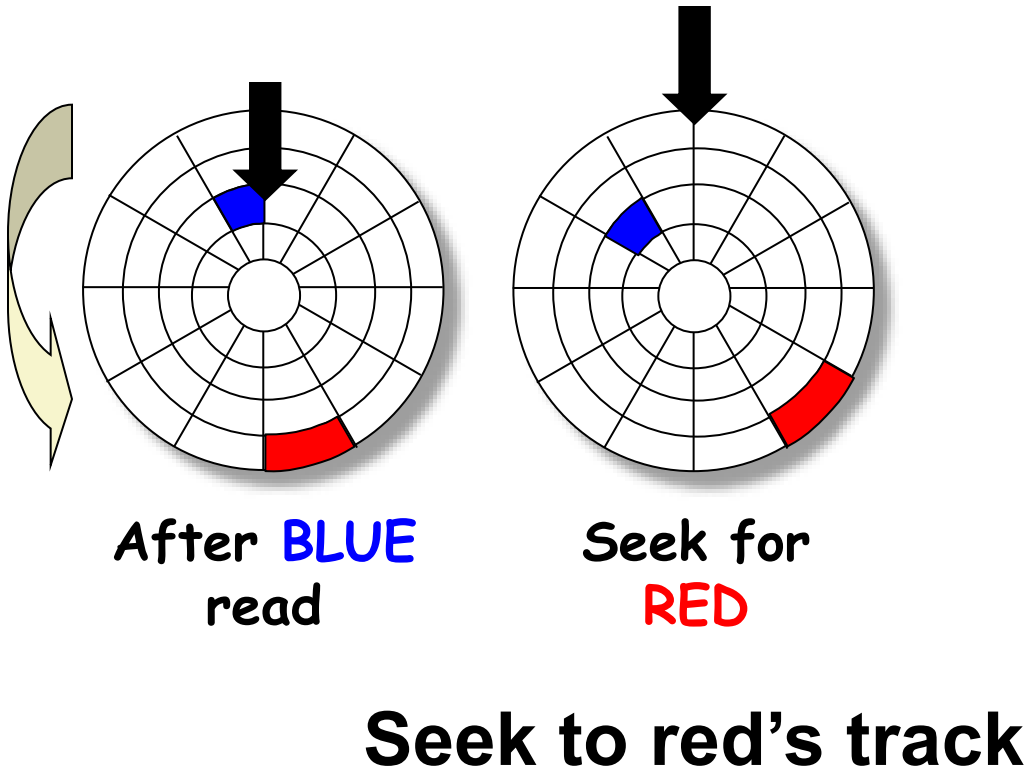
Disk Access - Read



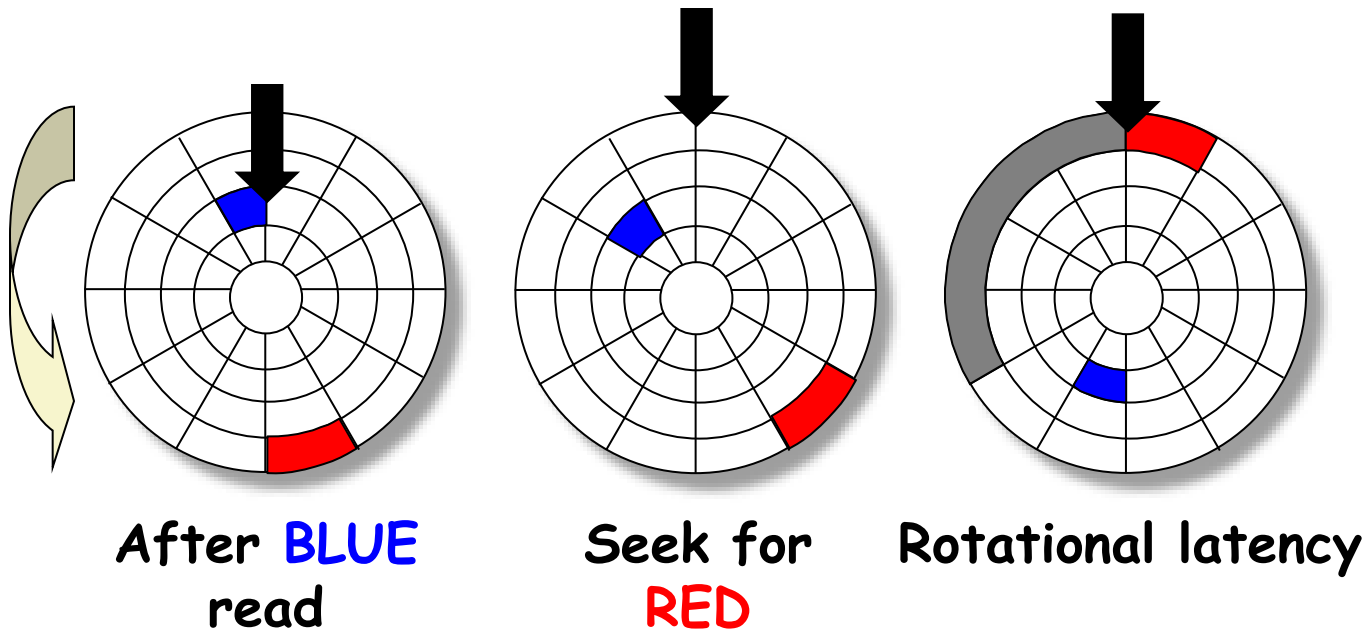
After **BLUE**
read

Red request scheduled next

Disk Access - Seek

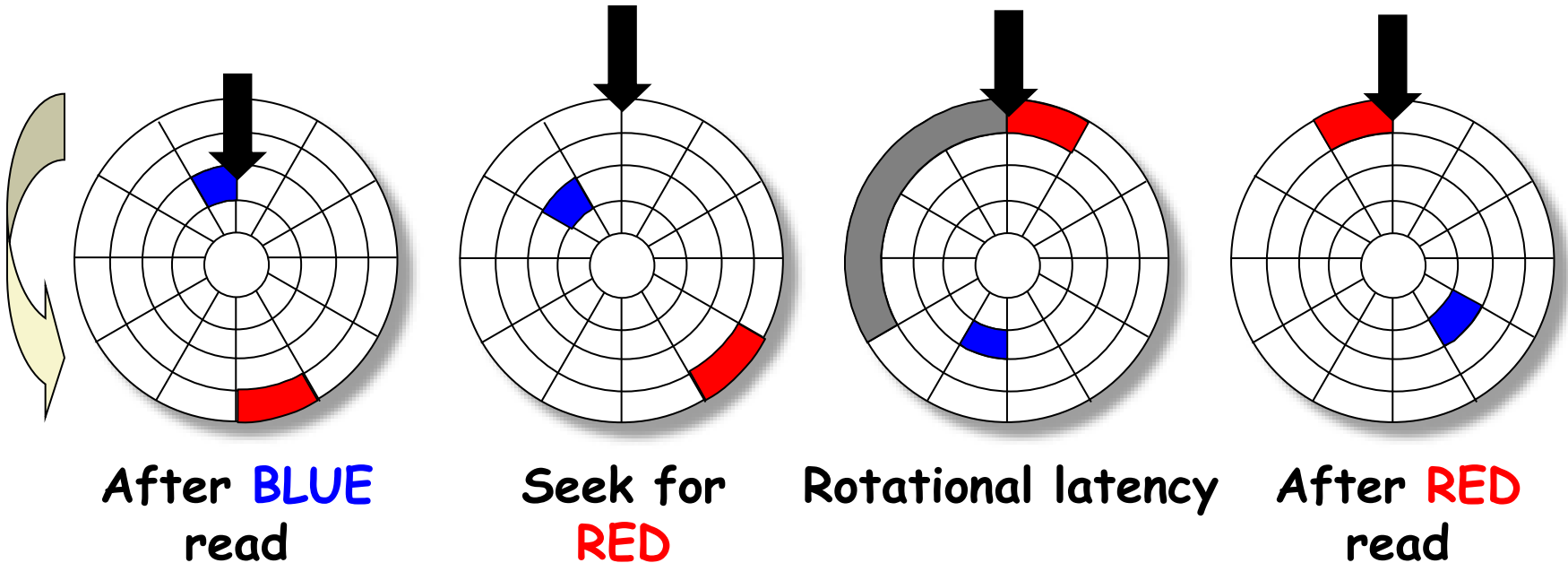


Disk Access - Rotational Latency



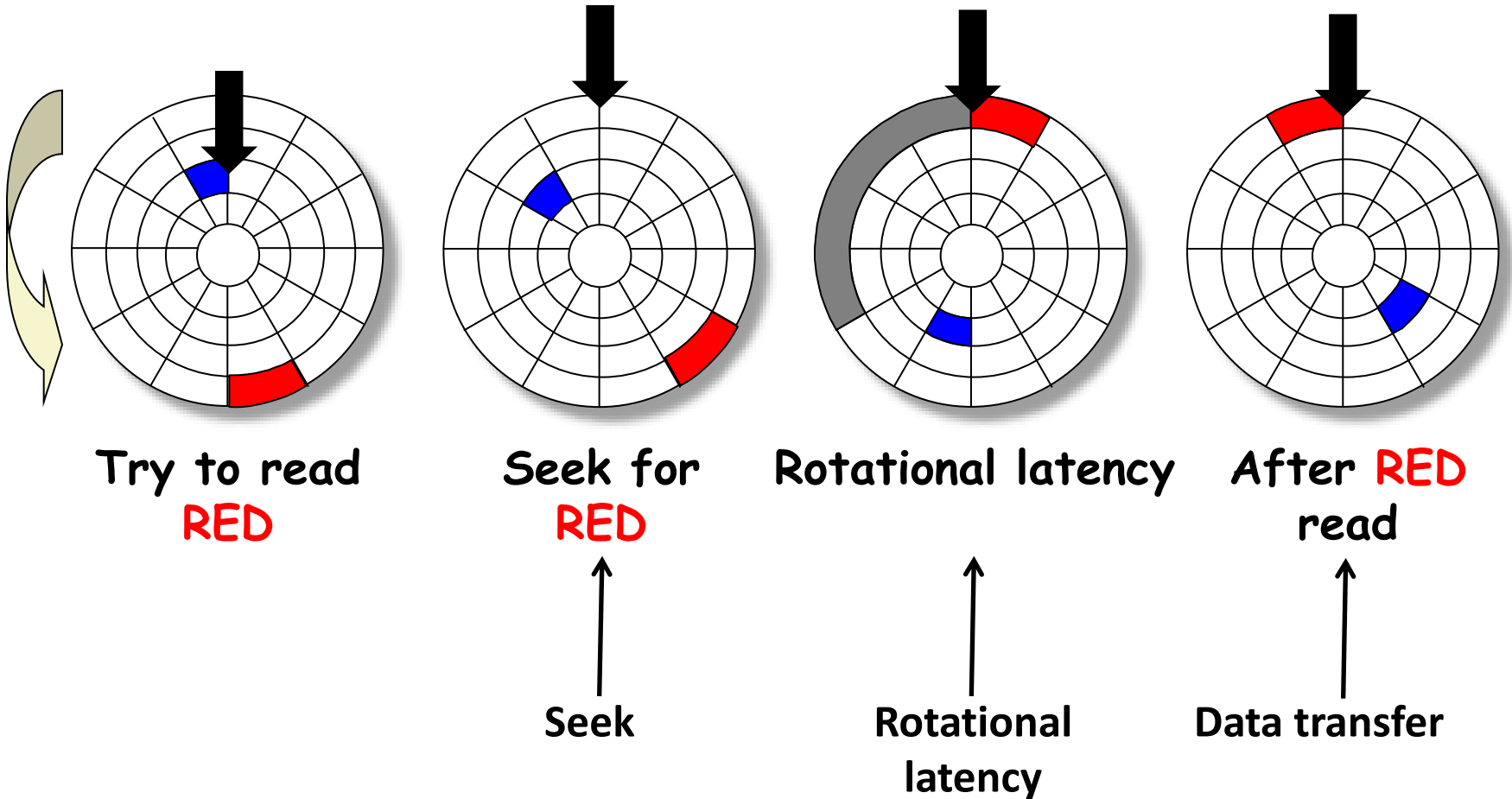
Wait for red sector to rotate around

Disk Access - Read



Complete read of red

Disk Access - Service Time Components



Disk access time

- Average time to access some target sector approximated by
 - $T_{\text{access}} = T_{\text{avg seek}} + T_{\text{avg rotation}} + T_{\text{avg transfer}}$
- Seek time
 - Time to position heads over cylinder containing target sector.
 - Typical $T_{\text{avg seek}} = 9 \text{ ms}$

Disk access time

- Rotational latency
 - Time waiting for first bit of target sector to pass under r/w head.
 - $T_{\text{avg rotation}} = 1/2 \times 1/\text{RPMs} \times 60 \text{ sec}/1 \text{ min}$
- Transfer time
 - Time to read the bits in the target sector.
 - $T_{\text{avg transfer}} = 1/(\text{avg \# sectors/track}) \times 1/\text{RPM} \times 60 \text{ secs}/1 \text{ min.}$

Disk access time example

Example:

Rotational rate 7,200 RPM

$T_{\text{avg seek}}$ 9 ms

Avg # sectors/track 400

$$\begin{aligned} T_{\text{avg rotation}} &= 1/2 * (60\text{secs}/7200\text{RPM}) * 1000\text{ms/sec} \\ &= 4\text{ms} \end{aligned}$$

$$\begin{aligned} T_{\text{avg transfer}} &= 1/400\text{secs/track} * 60\text{sec}/7200\text{RPM} * 1000\text{ms/sec} \\ &= 0.02 \text{ ms} \end{aligned}$$

$$T_{\text{access}} = 9 \text{ ms} + 4 \text{ ms} + 0.02 \text{ ms}$$

Disk access time example

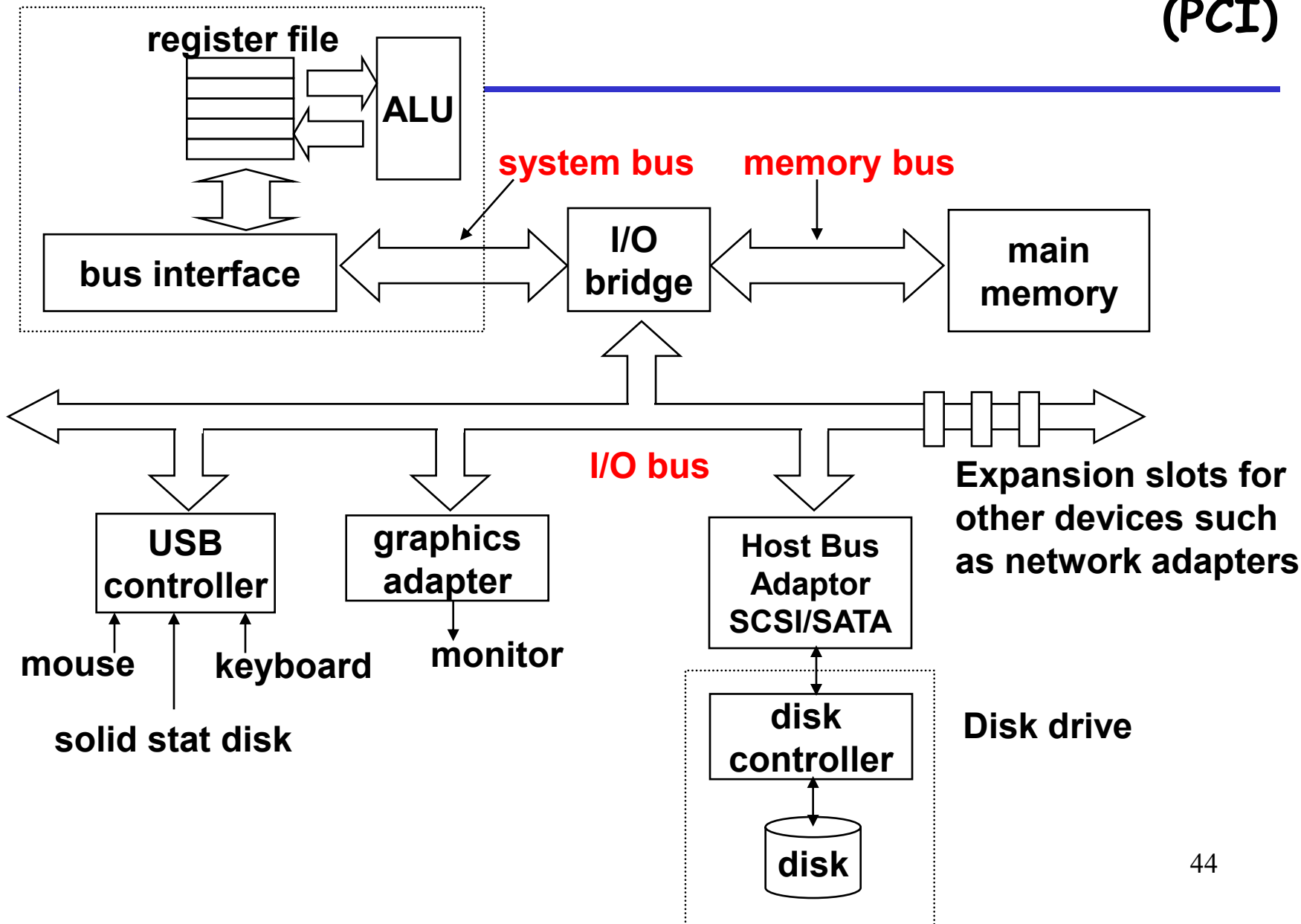
- Important points
 - Access time dominated by **seek time** and **rotational latency**, about 10ms for 512 bytes
 - First bit in a sector is the most expensive, the rest are free
 - SRAM access time is about 4 ns/64-bit word
 - DRAM is about 60 ns/64-bit word
 - Disk is about 40,000 times slower than SRAM
 - Disk is about 2,500 times slower than DRAM

Logical disk blocks

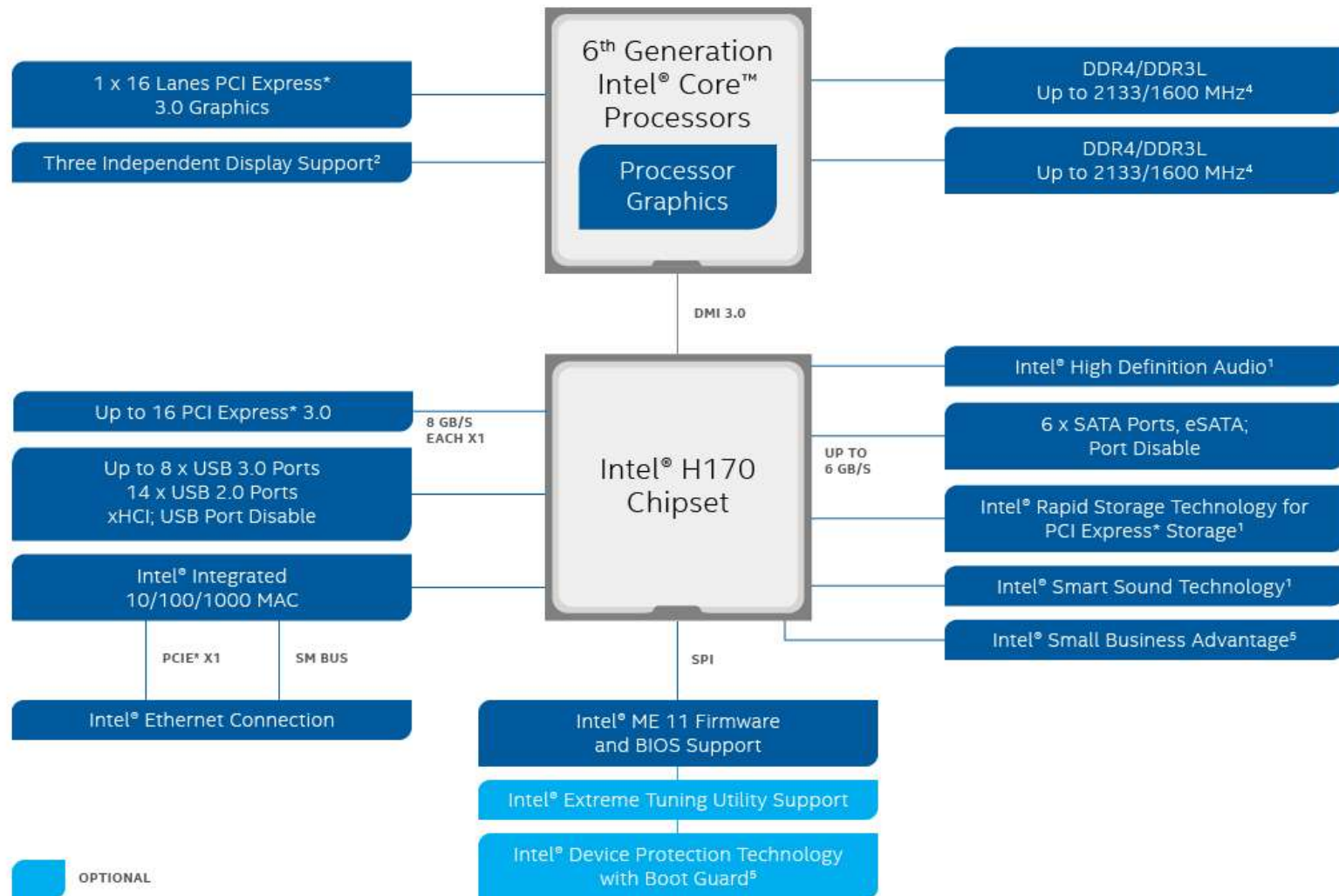
- Modern disks present a simpler **abstract** view of the complex sector geometry:
 - The set of available sectors is modeled as a sequence of B sector-size **logical blocks** $(0, 1, \dots, B-1)$
- Mapping between **logical blocks** and actual **(physical) sectors**
 - Maintained by hardware/firmware device called **disk controller**
 - Converts requests for logical blocks into **(surface, track, sector)** triples.

Peripheral Component Interconnect (PCI)

CPU chip



INTEL® H170 CHIPSET BLOCK DIAGRAM

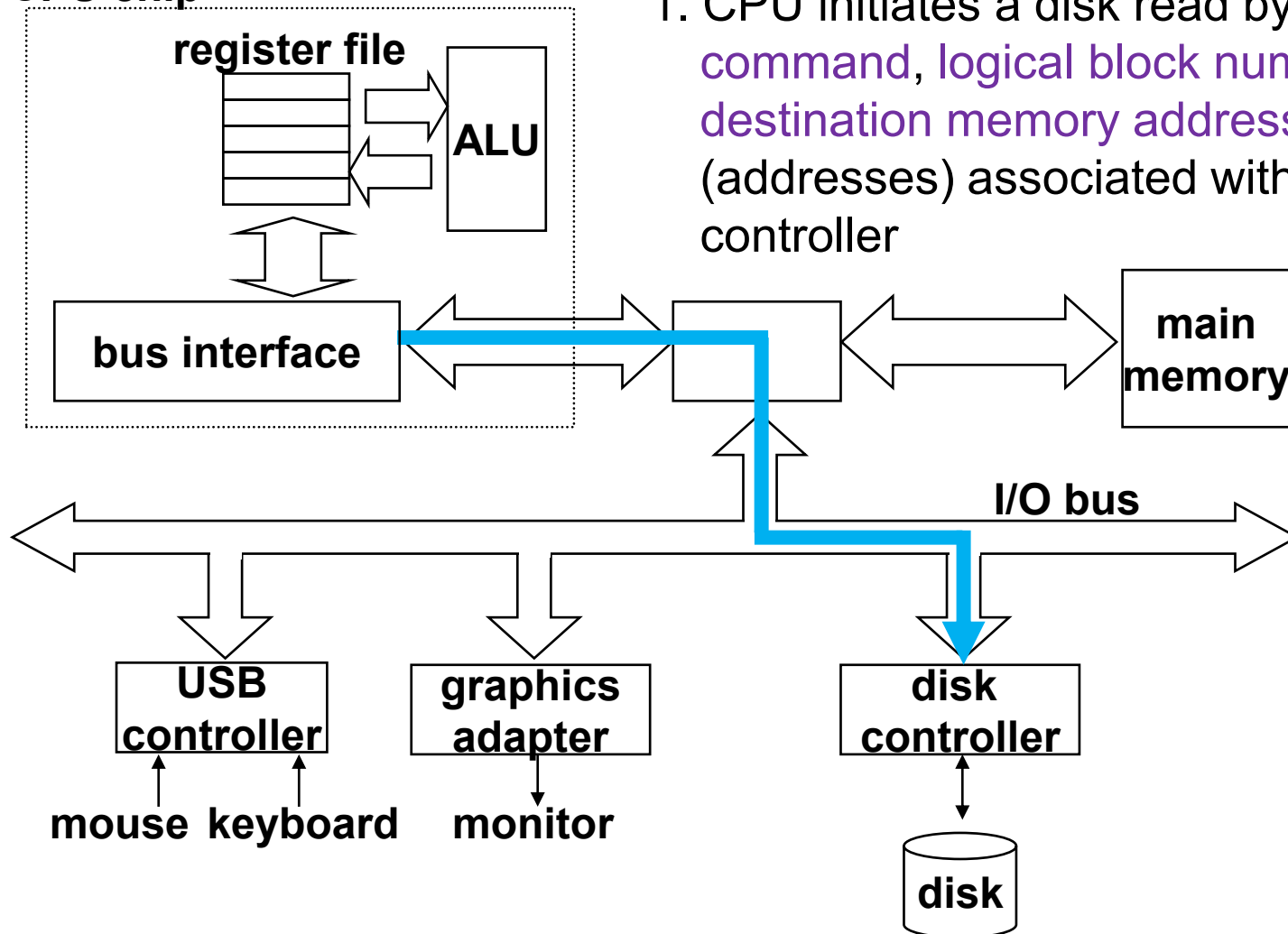


Memory-mapped I/O

- There are processors in controllers and adapters
- I/O Port
 - A reserved address in the address space
 - Some registers in the above processors are mapped to these addresses
- Each device is associated with
 - One or more ports when it is attached to the bus
- The CPU is able to communicate with an I/O device via its I/O ports

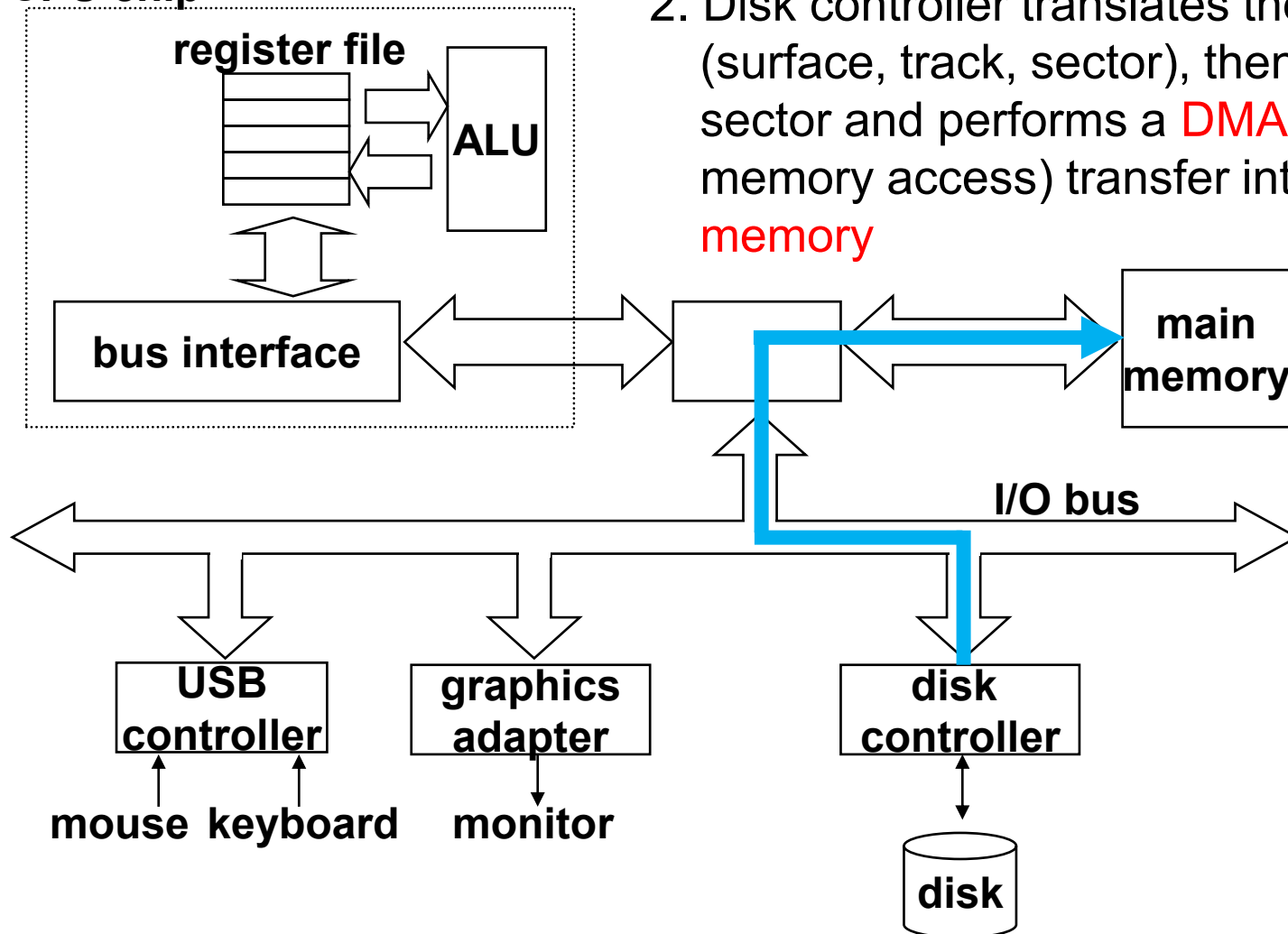
Reading a disk block

CPU chip



Reading a disk sector

CPU chip

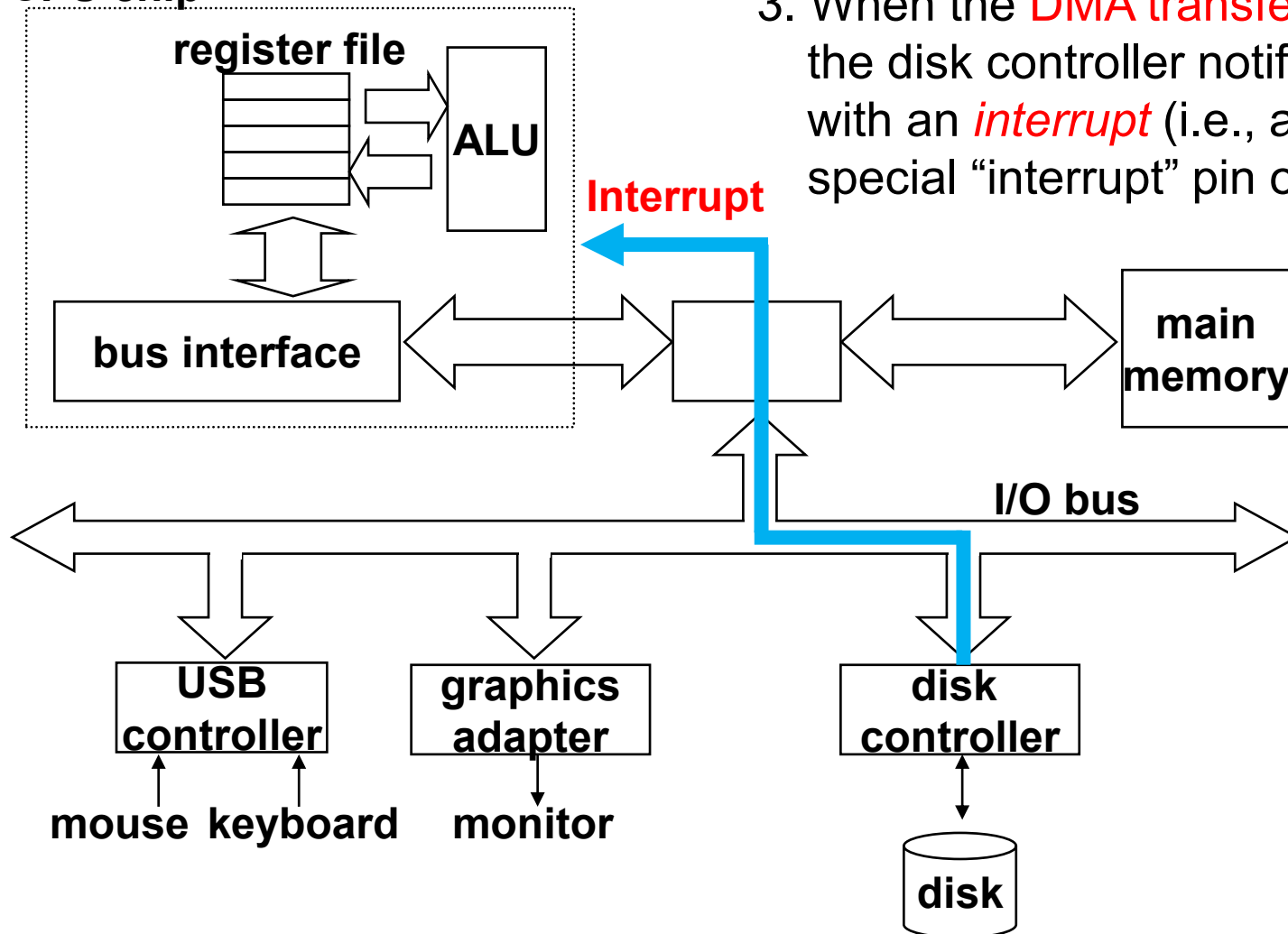


DMA

- Direct memory access
- A device performs a read or write bus transaction on its own, without any involvement of the CPU
- The transfer of data is known as a DMA transfer

Reading a disk block

CPU chip



3. When the **DMA transfer** completes, the disk controller notifies the CPU with an **interrupt** (i.e., asserts a special “interrupt” pin on the CPU)

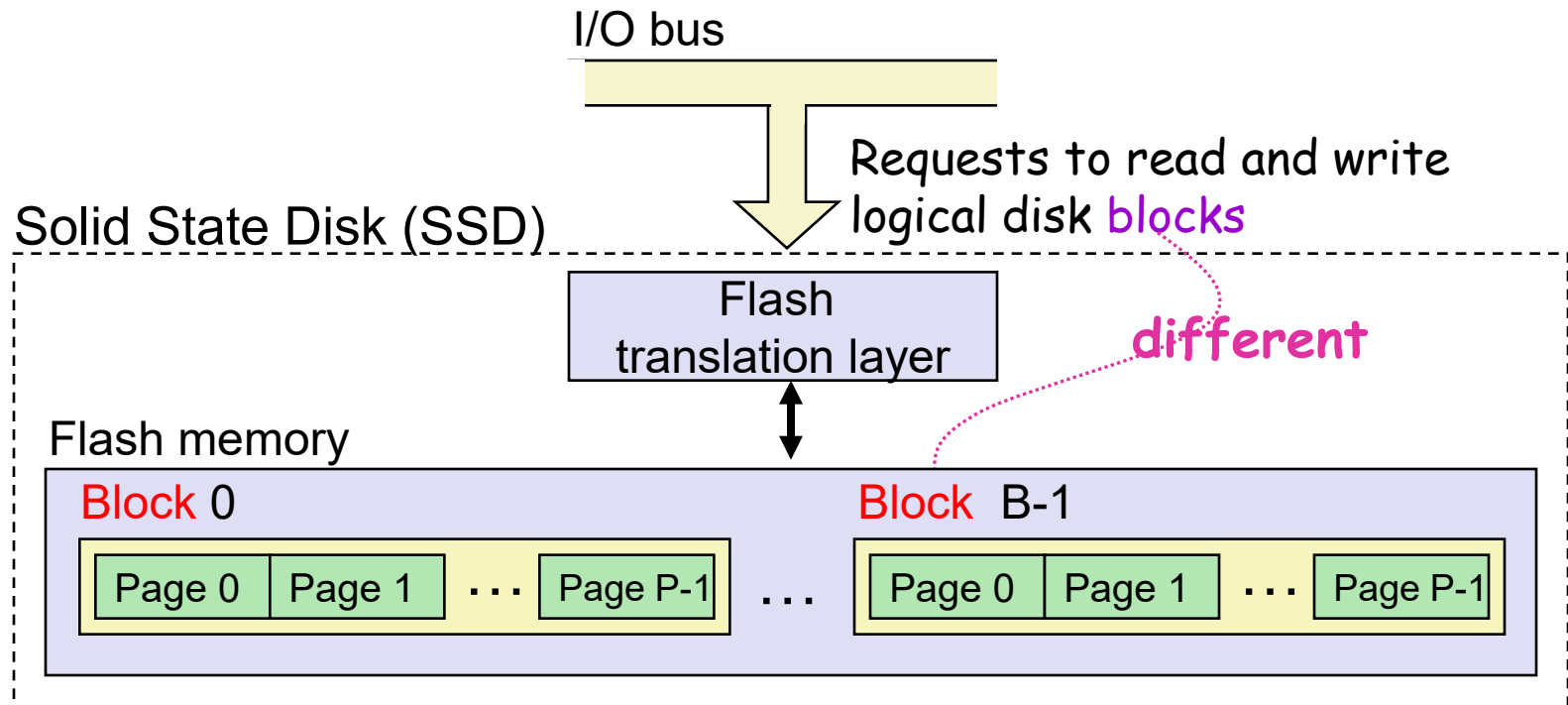
Interrupt

- I/O devices trigger interrupts by signaling a **pin** on the processor chip
 - network adapters, disk controllers, timer chips
- I/O devices place a number onto the system bus
 - identifies the device that caused the interrupt
- Cause the CPU stop what it is currently working on, and jump to an operating system routine

Solid State Disk (SSD)

- Solid state disk (SSD) is a storage technology, based on flash memory
 - SSD package plugs into a standard disk slot on the I/O bus (typically USB or SATA or PCIe)
 - Flash translation layer = disk controller

Solid State Disk (SSD)



- **Pages**: 512B to 4KB, **Blocks**: 32 to 128 pages
- Data read/written in units of **pages**.
- Page can be **written** only after its block has been **erased**
- A block **wears out** after 100,000 repeated writes.

SSD Performance Characteristics

- Benchmark of Samsung 970 EVO Plus

<https://ssd.userbenchmark.com/SpeedTest/711305/Samsung-SSD-970-EVO-Plus-250GB>

Sequential read throughput	2,126 MB/s	Sequential write tput	1,880 MB/s
Random read throughput	140 MB/s	Random write tput	59 MB/s

- Sequential access faster than random access
 - Common theme in the memory hierarchy
- Random writes are somewhat slower
 - Erasing a block takes a long time (~1 ms).
 - Modifying a block page requires all other pages to be copied to new block.
 - Flash translation layer allows accumulating series of small writes before doing block write.

SSD Tradeoffs vs Rotating Disks

- Advantages
 - No moving parts → faster, less power, more rugged
- Disadvantages
 - Have the potential to wear out
 - Mitigated by “wear leveling logic” in flash translation layer
 - E.g. Samsung 970 EVO Plus guarantees 600 writes/byte of writes before they wear out
 - Controller migrates data to minimize wear level
 - In 2019, about 4 times more expensive per byte
 - And, relative cost will keep dropping
- Applications
 - Smartphones, laptops
 - Increasingly common in desktops and servers

Storage Trends

SRAM

Metric	1985	1990	1995	2000	2005	2010	2015	2015:1985
\$/MB	2,900	320	256	100	75	60	25	116
access (ns)	150	35	15	3	2	1.5	1.3	115

DRAM

Metric	1985	1990	1995	2000	2005	2010	2015	2015:1985
\$/MB	880	100	30	1	0.1	0.06	0.02	44,000
access (ns)	200	100	70	60	50	40	20	10
typical size (MB)	0.256	4	16	64	2,000	8,000	16,000	62,500

Disk

Metric	1985	1990	1995	2000	2005	2010	2015	2015:1985
\$/GB	100,000	8,000	300	10	5	0.3	0.03	3,333,333
access (ms)	75	28	10	8	4	3	3	25
typical size (GB)	0.01	0.16	1	20	160	1,500	3000	1,500,000

CPU Clock Rates

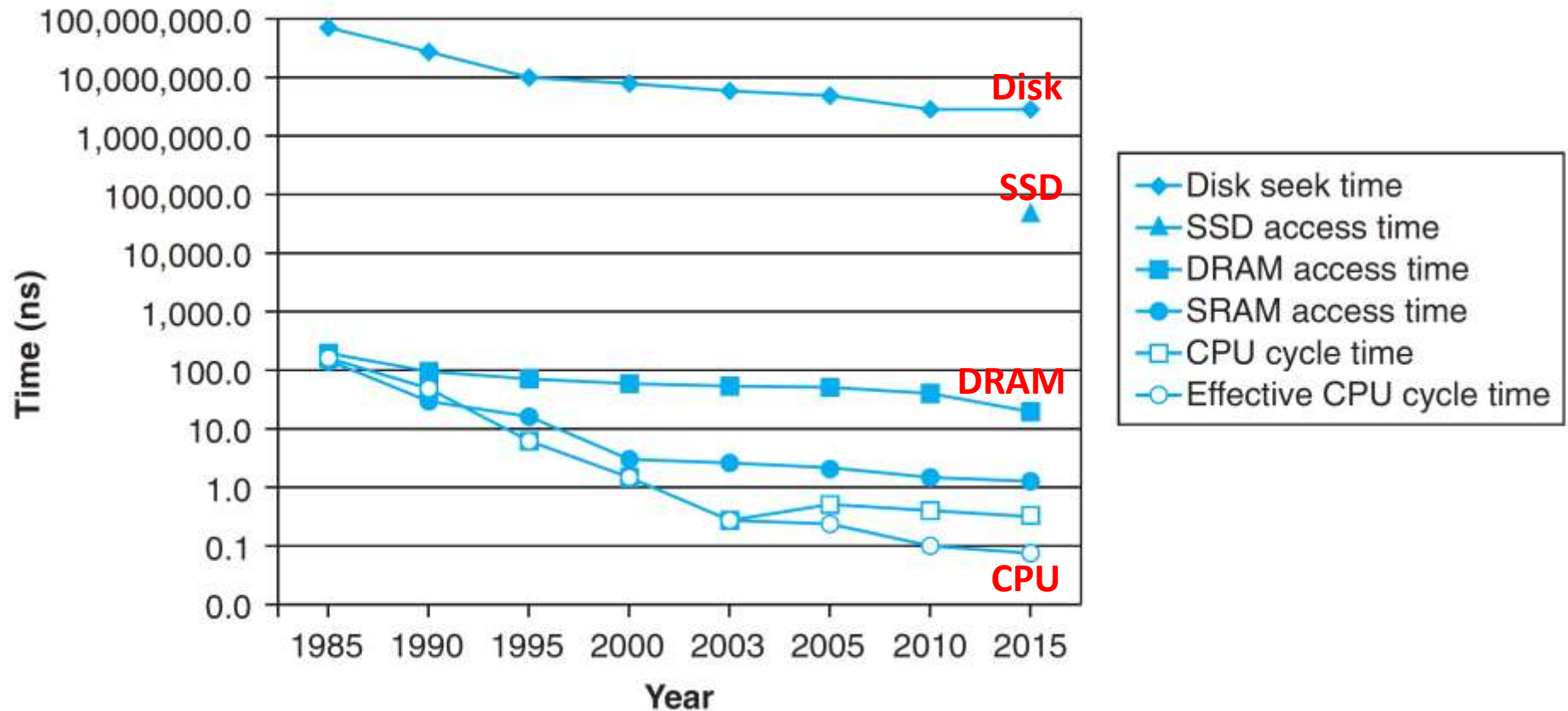
Inflection point in computer history
when designers hit the “Power Wall”



	1985	1990	1995	2000	2003	2005	2010	2015	2015:1985
CPU	80286	80386	Pent.	P-III	Pent.4	Core 2	Core i7(n)	Core i7(h)	---
Clock rate (MHz)	6	20	150	600	3,300	2,000	2,500	3,000	500
Cycle time (ns)	166	50	6	1.6	0.3	0.5	0.4	0.33	500
Cores	1	1	1	1	1	2	4	4	4
Effective cycle time (ns)	166	50	6	1.6	0.3	0.25	0.10	0.08	2,075

The CPU-Memory Gap

The gap **widens** between DRAM, disk, and CPU speeds.



The CPU-Memory Gap

The gap **widens** between DRAM, disk, and CPU speeds.

The key to bridging this CPU-Memory gap is a fundamental property of computer programs known as **locality**

